Lesson 8 Worksheet: Implementing the Navigator Class – ANSWER KEY

Part A: Navigator Class Design

1. Three attributes the Navigator needs:

Attribute	Type	Purpose
self.position	tuple	Stores the robot's current (row, col) on the grid
self.heading	string	Stores the compass direction the robot is facing (N, S, E, W)
self.drivetrain	DifferentialDrive	Controls the physical robot motors for turning and driving

2. Three main methods:

Method Name	Input	Output / Action
get_needed_direction	next_pos (tuple)	Returns the compass direction (N/S/E/W) to reach the next position
turn_to	direction (string)	Physically turns the robot to face the given direction
drive_path	path (list of tuples)	Drives the robot through every position in the path

3. Why does the Navigator store its own position instead of getting it from Manhattan?

The Navigator tracks its physical position after each step as it drives. Manhattan only plans paths – it doesn't know where the robot actually is after driving. The Navigator needs its own position to calculate which direction to face for the next step.

4. What object does the Navigator use to physically move the robot?

A DifferentialDrive object, obtained by calling `DifferentialDrive.get_default_differential_drive()`.

Part B: get_needed_direction() Exercises

Navigator Position	Next Position	row_diff	col_diff	Direction
(0, 0)	(1, 0)	1 - 0 = 1	0 - 0 = 0	S
(2, 1)	(1, 1)	1 - 2 = -1	1 - 1 = 0	N
(1, 2)	(1, 3)	0	1	E
(3, 3)	(3, 2)	0	-1	W
(0, 1)	(1, 1)	1	0	S
(2, 0)	(2, 1)	0	1	E
(3, 2)	(2, 2)	-1	0	N
(1, 3)	(1, 2)	0	-1	W

Part C: turn_to() Exercises

Current Heading	Needed Direction	Turn Type	Degrees	New Heading
N	E	Right	90	E
N	S	Reverse	180	S
E	E	None	0	E
E	N	Left	-90	N
S	W	Right	90	W
S	N	Reverse	180	N
W	S	Left	-90	S
W	E	Reverse	180	E
N	W	Left	-90	W

Current Heading	Needed Direction	Turn Type	Degrees	New Heading
E	W	Reverse	180	W

- How many required NO turn? 1 (E→E)
- How many required 180? 4 (N→S, S→N, W→E, E→W)

Part D: drive_path() Tracing

First path: `[(0, 0), (1, 0), (2, 0), (2, 1), (2, 2)]`

Starting position: (0, 0), Starting heading: N

Step (i)	next_pos	Needed Direction	Current Heading	Turn	Degrees	New Heading	New Position
1	(1, 0)	S	N	Reverse	180	S	(1, 0)
2	(2, 0)	S	S	None	0	S	(2, 0)
3	(2, 1)	E	S	Left	-90	E	(2, 1)
4	(2, 2)	E	E	None	0	E	(2, 2)

Final position: (2, 2), Final heading: E

Second path: `[(3, 3), (3, 2), (3, 1), (2, 1), (1, 1), (0, 1)]`

Starting position: (3, 3), Starting heading: E

Step (i)	next_pos	Needed Direction	Current Heading	Turn	Degrees	New Heading	New Position
1	(3, 2)	W	E	Reverse	180	W	(3, 2)
2	(3, 1)	W	W	None	0	W	(3, 1)
3	(2, 1)	N	W	Right	90	N	(2, 1)
4	(1, 1)	N	N	None	0	N	(1, 1)
5	(0, 1)	N	N	None	0	N	(0, 1)

Final position: (0, 1), Final heading: N

Part E: Code Completion

```
def __init__(self, start, heading):
    self.position = start
    self.heading = heading
    self.drivetrain = DifferentialDrive.get_default_differential_drive()
```

Blanks: start, heading

```
def get_needed_direction(self, next_pos):
    row_diff = next_pos[0] - self.position[0]
    col_diff = next_pos[1] - self.position[1]
    if row_diff == 1:
        return "S"
    elif row_diff == -1:
        return "N"
    elif col_diff == 1:
        return "E"
    elif col_diff == -1:
        return "W"
```

Blanks: 0, 0, 1, 1, S, N, E, W

```
def turn_to(self, direction):
    right_turns = {"N": "E", "E": "S", "S": "W", "W": "N"}
    left_turns = {"N": "W", "W": "S", "S": "E", "E": "N"}

    if self.heading == direction:
        pass
    elif right_turns[self.heading] == direction:
        self.drivetrain.turn(90)
        self.heading = direction
    elif left_turns[self.heading] == direction:
        self.drivetrain.turn(-90)
        self.heading = direction
    else:
        self.drivetrain.turn(180)
        self.heading = direction
```

Blanks: W, N, E, N, self.heading, 90, direction, self.heading, -90, direction, 180, direction

---

```
def drive_path(self, path):
    for i in range(1, len(path)):
        next_pos = path[i]
        direction = self.get_needed_direction(next_pos)
        self.turn_to(direction)
        self.drivetrain.straight(20)
        self.position = next_pos
```

Blanks: 1, i, next_pos, direction, next_pos

Why does the loop start at range(1, ...) instead of range(0, ...)?

Because path[0] is the robot's current position – it's already there. If the loop started at 0, the robot would try to "navigate" to where it already is, which would produce a zero-distance direction calculation that doesn't make sense.

Reflection

Why is it better to have two separate classes (Manhattan and Navigator) instead of one?

Separation of concerns makes each class simpler and easier to test. The Manhattan class handles pure math – computing paths from coordinates – and can be tested entirely on screen without a robot. The Navigator class handles physical movement – turning and driving – and only needs to know about one step at a time. If a path is wrong, you know the bug is in Manhattan. If the robot turns the wrong way, you know the bug is in Navigator. You can also reuse Manhattan with a different robot or test Navigator with a hand-written path. Combining them would make debugging much harder because you wouldn't know whether the problem is in the planning or the driving.